

Usando zip *e enumerate*

Duas funções que aparecem em quase todo código Python: `enumerate` numera os itens enquanto você percorre, e `zip` percorre várias sequências lado a lado. As duas devolvem iteradores (aula 02) e acabam com o `range(len(...))`. Nesta aula: `enumerate` com `start`, `zip` com vários iteráveis, o que acontece com tamanhos diferentes (`strict` e `zip_longest`), montar dicionários, "desfazer" com `zip(*...)` e as duas juntas.

`enumerate(it, start=1)`

Adeus `range(len())`

`zip(a, b, c)`

`strict=True`

`zip_longest`

`dict(zip())`

`zip(*pares)`

Formação Claude Code 2026 · Baseado em docs.python.org (Built-in Functions: enumerate, zip; itertools.zip_longest; Tutorial: Looping Techniques, Unpacking Argument Lists), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em [exemplos/](#) · Saídas reais

Índice e item *juntos*

enumerate_basico.py

```
tarefas = ["estudar", "treinar", "revisar"]

for i, tarefa in enumerate(tarefas):
    print(i, tarefa)

for n, tarefa in enumerate(tarefas, start=1):    # começa em 1
    print(f"{n}. {tarefa}")

print(list(enumerate("ab")))
```

SAÍDA

```
0 estudar
1 treinar
2 revisar
1. estudar
2. treinar
3. revisar
[(0, 'a'), (1, 'b')]
```

- ✓ Cada volta entrega uma tupla `(índice, item)`, desempacotada em `i, tarefa`
- ✓ `start=1` muda o primeiro número: perfeito para listas numeradas
- ✓ Funciona com qualquer iterável: lista, string, arquivo, gerador

Adeus, *range(len())*

sem_range_len.py

```
cores = ["azul", "verde", "roxo"]

# Jeito antigo: índice para pegar o item
for i in range(len(cores)):
    if cores[i] == "verde":
        print("verde na posição", i)

# Jeito pythônico: índice e item juntos
for i, cor in enumerate(cores):
    if cor == "verde":
        print("verde na posição", i)
```

SAÍDA

```
verde na posição 1
verde na posição 1
```

- ✓ O resultado é o mesmo; a segunda forma não precisa de `cores[i]`
- ✓ Menos chance de errar o índice e mais fácil de ler
- ✓ Só quer os itens? Nem precisa do `enumerate`: `for cor in cores`

Ao ver `for i in range(len(x))`, pergunte: dá para usar `enumerate` ou `zip`?

Várias listas *lado a lado*

zip_basico.py

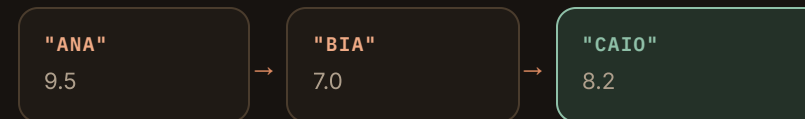
```
nomes = ["Ana", "Bia", "Caio"]
notas = [9.5, 7.0, 8.2]
turmas = ["A", "B", "A"]

for nome, nota in zip(nomes, notas):
    print(f"{nome}: {nota}")

print(list(zip(nomes, notas, turmas))) # três de uma vez
```

SAÍDA

```
Ana: 9.5
Bia: 7.0
Caio: 8.2
[('Ana', 9.5, 'A'), ('Bia', 7.0, 'B'), ('Caio', 8.2, 'A')]
```



- ✓ `zip` junta o 1º de cada, depois o 2º de cada, e assim por diante
- ✓ Aceita quantos iteráveis você quiser: cada volta traz uma tupla com um item de cada
- ✓ Como é um iterador, use `list()` para ver tudo de uma vez

Quando uma lista *é menor*

tamanhos.py

```
from itertools import zip_longest

nomes = ["Ana", "Bia", "Caio"]
notas = [9.5, 7.0]                # faltou uma nota

print(list(zip(nomes, notas)))    # para no menor
print(list(zip_longest(nomes, notas, fillvalue="-")))

try:
    list(zip(nomes, notas, strict=True)) # exige mesmo tamanho
except ValueError as erro:
    print("Erro:", erro)
```

SAÍDA

```
[('Ana', 9.5), ('Bia', 7.0)]
[('Ana', 9.5), ('Bia', 7.0), ('Caio', '-')]
Erro: zip() argument 2 is shorter than argument 1
```

FORMA

COM TAMANHOS DIFERENTES

`zip(a, b)`

para no menor, sem avisar

`zip_longest(a, b,
fillvalue=x)`

vai até o maior, completando com

`x`

`zip(a, b, strict=True)`

lança `ValueError`

O corte silencioso esconde dados perdidos: o Caio ficou sem nota e ninguém avisou. Quando as listas **deveriam** ter o mesmo tamanho, use `strict=True`.

dict(zip()): *chaves e valores*

com_dict.py

```
campos = ["nome", "idade", "cidade"]
valores = ["Ana", 30, "Recife"]

pessoa = dict(zip(campos, valores))
print(pessoa)

precos = {"café": 4.5, "pão": 0.8}
invertido = dict(zip(precos.values(), precos.keys()))
print(invertido)
```

SAÍDA

```
{'nome': 'Ana', 'idade': 30, 'cidade': 'Recife'}
{4.5: 'café', 0.8: 'pão'}
```

- ✓ `dict()` aceita pares; `zip` produz pares: dois em uma linha
- ✓ Uso típico: cabeçalho de planilha + uma linha de valores = um registro
- ✓ Trocando a ordem, o dicionário se inverte (valores viram chaves)

Ao inverter, valores repetidos viram chaves repetidas, e só a última sobrevive.

Desfazer pares *e transpor*

desfazer.py

```
pares = [("Ana", 9.5), ("Bia", 7.0), ("Caio", 8.2)]

nomes, notas = zip(*pares)          # o * espalha os pares
print(nomes)
print(notas)

tabela = [[1, 2, 3],
           [4, 5, 6]]
print(list(zip(*tabela)))           # linhas viram colunas
```

SAÍDA

```
('Ana', 'Bia', 'Caio')
(9.5, 7.0, 8.2)
[(1, 4), (2, 5), (3, 6)]
```

- ✓ `zip(*pares)` é o caminho inverso: separa uma lista de pares em duas sequências
- ✓ O `*` espalha os itens como argumentos (aula 20 do módulo 37)
- ✓ Com uma tabela (lista de linhas), `zip(*tabela)` devolve as colunas

Ranking *da turma*

Enunciado: com as listas de alunos e das notas de duas provas, calcule a média de cada um e mostre o ranking numerado, da maior para a menor média.

Uma resposta possível

```
alunos = ["Ana", "Bia", "Caio", "Davi"]
p1 = [8.0, 6.5, 9.0, 7.0]
p2 = [9.0, 7.5, 8.5, 5.0]

medias = [(a + b) / 2 for a, b in zip(p1, p2, strict=True)]
ranking = sorted(zip(alunos, medias), key=lambda t: t[1],
                  reverse=True)

for posicao, (nome, media) in enumerate(ranking, start=1):
    print(f"{posicao}º {nome:<5} {media:.2f}")
```

SAÍDA

```
1º Caio  8.75
2º Ana   8.50
3º Bia   7.00
4º Davi  6.00
```

- ✓ `zip(p1, p2, strict=True)` garante uma nota de cada prova por aluno
- ✓ `zip(alunos, medias)` monta os pares; `enumerate(..., start=1)`, as posições
- ✓ `(nome, media)` entre parênteses desempacota o par dentro da tupla do `enumerate`

Teste o que *ficou*

1. `list(enumerate("xy", start=1))` vale:

A) [(0, "x"), (1, "y")]

B) [(1, "x"), (2, "y")]

C) ["x", "y"]

D) [1, 2]

B.

2. `list(zip([1, 2, 3], "ab"))` vale:

A) [(1, "a"), (2, "b")]

B) [(1, "a"), (2, "b"), (3, None)]

C) Erro

D) [1, 2, 3, "a", "b"]

A. Para no menor.

3. Para dar erro quando os tamanhos forem diferentes:

A) `zip_longest`

B) `zip(..., strict=True)`

C) `enumerate`

D) `zip(*...)`

B.

4. `dict(zip(["a", "b"], [1, 2]))` vale:

A) {"a": 1, "b": 2}

B) {1: "a", 2: "b"}

C) [("a", 1), ("b", 2)]

D) Erro

A.

Numerar e *andar lado a lado*

`enumerate` entrega índice e item (com `start` para escolher o primeiro número) e `zip` percorre várias sequências juntas, parando na menor. Use `strict=True` quando os tamanhos devem bater, `zip_longest` quando um pode faltar, `dict(zip())` para montar registros e `zip(*...)` para desfazer pares ou transpor tabelas.

`enumerate`

`zip`

`strict=True`

`zip_longest`

`dict(zip())`

`zip(*)`

Próxima aula → 16 · Manipulação de Arquivos I

Fontes: docs.python.org/3/library/functions.html (`enumerate`, `zip`); docs.python.org/3/library/itertools.html#itertools.zip_longest; docs.python.org/3/tutorial/datastructures.html#looping-techniques; docs.python.org/3/tutorial/controlflow.html#unpacking-argument-lists, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.